

Informe de Resultados

Benchmark de APIs con PostgreSQL

Evaluación Comparativa de 4 Stacks Tecnológicos

Stacks evaluados:

Stack	Tecnologías
Python	Flask + psycopg2 + Gunicorn
Node.js	NestJS + pg (driver nativo)
Java	Spring Boot + JdbcTemplate
Go	net/http + pgx (driver nativo)

Arquitectura: **Clean Architecture**

Herramienta de carga: **Apache JMeter 5.6.3**

Base de datos: **PostgreSQL 16 (Docker)**

25 de mayo de 2026

Índice

1. Introducción y Metodología	2
1.1. Endpoint evaluado	2
1.2. Escenarios de carga (JMeter)	2
1.3. Métricas registradas	2
2. Resultados Individuales por Stack	3
2.1. Flask (Python)	3
2.2. Go (stdlib + pgx)	3
2.3. NestJS (Node.js)	4
2.4. Spring Boot (Java)	4
3. Tabla Comparativa — Entrega Grupal	5
3.1. Comparación de rendimiento por escenario	5
3.1.1. Baja carga (10 usuarios)	5
3.1.2. Media carga (50 usuarios)	5
3.1.3. Alta carga (100 usuarios)	5
3.1.4. Estrés (150 usuarios)	5
4. Consumo del Sistema (CPU / RAM)	6
5. Evidencias	7
6. Conclusiones	8
6.1. Rendimiento	8
6.2. Eficiencia	8
6.3. Recomendación para producción	8
7. Anexos	10
7.1. Comandos utilizados	10
7.2. Estructura del repositorio	10

1. Introducción y Metodología

El presente informe documenta los resultados de un benchmark comparativo entre cuatro stacks tecnológicos implementando el mismo microservicio: `POST /api/orders`. Cada stack sigue los principios de **Clean Architecture**, con separación de capas (Domain, Application, Adapter, Infrastructure) y uso de SQL nativo (sin ORMs).

1.1. Endpoint evaluado

- **URL:** `POST http://localhost:<puerto>/api/orders`
- **Request body:** JSON con `customerId` y lista de `items`
- **Response:** JSON con `orderId`, `totalAmount`, `itemsCount`, `createdAt`
- **Base de datos:** PostgreSQL 16 en contenedor Docker independiente por stack

1.2. Escenarios de carga (JMeter)

Escenario	Usuarios	Duración (s)	Ramp-up (s)
Baja carga	10	60	2
Media carga	50	120	5
Alta carga	100	180	10
Estrés	150	240	15

Cuadro 1: Configuración de Thread Groups en JMeter

1.3. Métricas registradas

Por cada escenario se registraron:

1. **Average Response Time** (ms): tiempo promedio de respuesta
2. **Percentil 95** (ms): latencia en el percentil 95
3. **Throughput** (req/s): solicitudes procesadas por segundo
4. **Error rate** (%): porcentaje de requests fallidos
5. **Observaciones:** cuellos de botella, timeouts, comportamiento anómalo

2. Resultados Individuales por Stack

2.1. Flask (Python)

Escenario	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Baja carga (10)	17,1	32	563,3	0,00 %
Media carga (50)	86,3	146	566,3	0,00 %
Alta carga (100)	150,4	249	646,4	0,00 %
Estrés (150)	292,9	508	496,3	0,00 %

Cuadro 2: Resultados de rendimiento — Flask (Python)

Observaciones:

- El servidor Gunicorn con 4 workers manejó bien las cargas baja y media.
- Bajo alta carga (100 usuarios), la latencia promedio aumentó a 150 ms.
- En el escenario de estrés (150 usuarios), el throughput disminuyó de 646 a 496 req/s, indicando saturación del pool de workers.
- No se registraron errores HTTP ni timeouts en ningún escenario.
- El GIL de Python limita el paralelismo real a nivel de proceso; el uso de múltiples workers mitiga pero no elimina el cuello de botella.

2.2. Go (stdlib + pgx)

Escenario	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Baja carga (10)	4,5	11	2111,4	0,00 %
Media carga (50)	21,3	35	2274,8	0,00 %
Alta carga (100)	40,6	78	2384,7	0,00 %
Estrés (150)	39,7	52	3624,6	0,00 %

Cuadro 3: Resultados de rendimiento — Go (stdlib)

Observaciones:

- Go demostró el mejor rendimiento absoluto en todos los escenarios.
- La latencia promedio se mantuvo por debajo de 42 ms incluso bajo 150 usuarios concurrentes.
- El throughput escaló de 2111 req/s (baja) a 3624 req/s (estrés), mostrando una curva de escalabilidad casi lineal.
- No se registraron errores ni fallas de conexión a la base de datos.
- El modelo de goroutines + pgxpool permite un manejo eficiente de concurrencia sin el overhead de threads del sistema operativo.

2.3. NestJS (Node.js)

Nota: Los benchmarks para este stack no fueron ejecutados en esta sesión. Las tablas deben completarse ejecutando:

```
cd nodejs-nestjs && docker compose up -d --build
./scripts/run-load-test.sh nodejs 5002 low
./scripts/run-load-test.sh nodejs 5002 medium
./scripts/run-load-test.sh nodejs 5002 high
./scripts/run-load-test.sh nodejs 5002 stress
```

2.4. Spring Boot (Java)

Nota: Los benchmarks para este stack no fueron ejecutados en esta sesión. Las tablas deben completarse ejecutando:

```
cd java-springboot && docker compose up -d --build
./scripts/run-load-test.sh java 5003 low
./scripts/run-load-test.sh java 5003 medium
./scripts/run-load-test.sh java 5003 high
./scripts/run-load-test.sh java 5003 stress
```

3. Tabla Comparativa — Entrega Grupal

3.1. Comparación de rendimiento por escenario

3.1.1. Baja carga (10 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput	% Error
Flask (Python)	17,1	32	563,3	0,00 %
Go (stdlib)	4,5	11	2111,4	0,00 %
NestJS (Node)	—	—	—	—
Spring Boot (Java)	—	—	—	—

Cuadro 4: Baja carga — Comparativa

3.1.2. Media carga (50 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput	% Error
Flask (Python)	86,3	146	566,3	0,00 %
Go (stdlib)	21,3	35	2274,8	0,00 %
NestJS (Node)	—	—	—	—
Spring Boot (Java)	—	—	—	—

Cuadro 5: Media carga — Comparativa

3.1.3. Alta carga (100 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput	% Error
Flask (Python)	150,4	249	646,4	0,00 %
Go (stdlib)	40,6	78	2384,7	0,00 %
NestJS (Node)	—	—	—	—
Spring Boot (Java)	—	—	—	—

Cuadro 6: Alta carga — Comparativa

3.1.4. Estrés (150 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput	% Error
Flask (Python)	292,9	508	496,3	0,00 %
Go (stdlib)	39,7	52	3624,6	0,00 %
NestJS (Node)	—	—	—	—
Spring Boot (Java)	—	—	—	—

Cuadro 7: Estrés — Comparativa

4. Consumo del Sistema (CPU / RAM)

Nota: Esta sección debe completarse con datos obtenidos mediante `docker stats` y `htop` durante la ejecución de cada prueba.

Stack	Escenario	CPU API	RAM API	CPU DB	RAM DB	Observaciones
Python	Baja (10)	—	—	—	—	(completar)
Python	Media (50)	—	—	—	—	(completar)
Python	Alta (100)	—	—	—	—	(completar)
Python	Estrés (150)	—	—	—	—	(completar)
Go	Baja (10)	—	—	—	—	(completar)
Go	Media (50)	—	—	—	—	(completar)
Go	Alta (100)	—	—	—	—	(completar)
Go	Estrés (150)	—	—	—	—	(completar)

Cuadro 8: Consumo de recursos por stack y escenario

5. Evidencias

Esta sección debe incluir las siguientes capturas de pantalla por cada stack y escenario:

1. **Summary Report** de JMeter
2. **Aggregate Report** de JMeter
3. **Pantallazo de JMeter** durante la ejecución de la carga
4. **docker ps** mostrando los contenedores activos
5. **htop** mostrando el consumo de CPU/RAM del host
6. **docker stats** mostrando métricas de los contenedores

Las capturas deben guardarse en `docs/evidencias/<stack>/<escenario>/`.

6. Conclusiones

6.1. Rendimiento

1. ¿Cuál stack tuvo mejor latencia promedio?

- Go fue el claro ganador con latencias promedio de 4,5 ms (baja carga) a 40,6 ms (alta carga).
- Python (Flask) presentó latencias significativamente mayores, alcanzando 292,9 ms bajo estrés.

2. ¿Cuál mantuvo el mejor throughput en cargas altas?

- Go escaló su throughput de 2111 req/s a 3624 req/s bajo estrés.
- Python alcanzó un pico de 646 req/s en alta carga pero decayó a 496 req/s bajo estrés, mostrando saturación.

3. ¿Qué lenguaje fue más estable?

- Ambos stacks (Python y Go) mantuvieron 0 % de errores en todos los escenarios.
- Go mostró mayor estabilidad en latencia, sin degradación abrupta.

6.2. Eficiencia

4. ¿Algún stack mostró errores bajo alta carga?

- No se registraron errores en Python ni Go.
- Los assertions de JMeter (status 200 + body con orderId) validaron la integridad de todas las respuestas.

5. ¿Qué puede explicar las diferencias de rendimiento?

- **Go:** modelo de goroutines ligero + pgxpool eficiente permite alta concurrencia sin overhead de threads del SO.
- **Python:** el GIL (Global Interpreter Lock) limita el paralelismo real dentro de un proceso. Aunque Unicorn usa 4 workers, el overhead de IPC y el modelo WSGI sincrónico generan latencias mayores bajo carga concurrente intensa.
- **Pool de conexiones:** ambos usan pool de 20 conexiones, pero Go reutiliza conexiones de forma más eficiente gracias a su runtime.

6.3. Recomendación para producción

6. ¿Cuál stack recomendarían para una empresa que requiere alto desempeño?

- **Go** es la recomendación para alto throughput y baja latencia. Su modelo de concurrencia y footprint de memoria reducido lo hacen ideal para microservicios de alta carga.

7. ¿Cuál stack es más fácil de implementar pero sacrifica rendimiento?

- **Python (Flask)** ofrece la curva de aprendizaje más suave y mayor cantidad de recursos educativos, pero su rendimiento es significativamente inferior bajo carga concurrente.

8. ¿Cuál stack ofrece un mejor balance entre sencillez y velocidad?

- **Go** ofrece un excelente balance: sintaxis moderna, compilación rápida, y rendimiento cercano a C/C++.
- **Node.js (NestJS)** podría ser una alternativa intermedia (pendiente de benchmark), ya que su modelo async/IO no bloqueante permite buen rendimiento con curva de aprendizaje moderada.

7. Anexos

7.1. Comandos utilizados

```
# Levantar un stack
cd python-flask && docker compose up -d --build

# Ejecutar benchmark
./scripts/run-load-test.sh python 5001 low

# Monitorear recursos
./scripts/monitor.sh python low

# Extraer métricas automáticamente
./scripts/extract-metrics.sh

# Generar reporte HTML de JMeter
# (automático al ejecutar run-load-test.sh)
```

7.2. Estructura del repositorio

```
api-benchmarking-jmeter/
  docs/
    INFORME-AUTO.md      <- Metricas extraidas de CSV
    evidencias/          <- Capturas de pantalla
    INFORME.pdf          <- Este documento
  results/               <- CSV y reportes HTML de JMeter
  scripts/
    run-load-test.sh     <- Wrapper JMeter
    monitor.sh           <- Docker stats
    extract-metrics.sh   <- Extrae metricas a Markdown
    run-full-benchmark.sh <- Orquestador completo
  shared/jmeter/         <- Test Plans (.jmx)
  python-flask/
  nodejs-nestjs/
  java-springboot/
  go-stdlib/
```

Documento generado automáticamente a partir de los resultados de JMeter.